

parser3 — руководство разработчика (деплой и эксплуатация)

Техническая документация по развёртыванию, конфигурации, API и обслуживанию парсера (parser3). Всё, что описано ниже, взято из репозитория ((Teketkom3026/parser3), ветка (exp/extractor)); внешние допущения помечены явно.

Скоуп. Документ описывает **только** (parser3). Сервис (parser2) (с ИИ) — это отдельная кодовая база вне этого репозитория, здесь не рассматривается. (parser3) и (parser3-dev) — один и тот же код в двух деплоях (см. ниже).

Архитектура

Два контейнера, поднимаются через Docker Compose:

Сервис	Тип	Стек	Внутр. порт
backend	API	FastAPI · Python 3.11 · uvicorn · httpx · Playwright/Chromium · aiosqlite (SQLite WAL)	(8000)
frontend	SPA	React 18 · Vite 5 · TypeScript, собранная статика под (nginx:alpine)	(80)

Backend монтируется под префикс (root_path) ((ROOT_PATH)), REST живёт под (/api/v1), WebSocket и служебные ручки ((/health), (/), (/docs), (/openapi.json)) — вне (/api/v1). Хранилище — SQLite в WAL-режиме (одна БД (parser3.db)), очереди/воркеры — внутрипроцессные ((TaskManager) + пул Chromium), внешних брокеров нет.

Конвейер обработки одного сайта:

fetch → find_pages → extract → normalize → classify → dedup

1. DNS-предпроверка (NXDOMAIN/таймаут → сразу ошибка, без браузера).
2. Фетч входной страницы; мёртвая «глубокая» ссылка → fallback на корень домена.
3. Извлечение реквизитов (ИНН/КПП/ОГРН/название/общий тел./email).
4. Поиск контактных страниц ((/kontakty), (/rukovodstvo), (/rekvizity) ...), скоринг URL, лимит (max_pages).
5. Извлечение ФИО + должность + личные контакты из HTML.
6. Нормализация: (petrovich) (падежи/пол), (pymorphy3) + (rapidfuzz) по (catalog/positions.yaml), соцсети.
7. Классификация листа + дедуп по (dedup_key).

Fetcher: сперва `(httpx)`; при connect/SSL-ошибке на HTTPS — ретрай по HTTP (кэш «http-only» хостов); детект SPA (короткий HTML / `(id="root")`) → fallback в Playwright Chromium. Браузер пропускается на 4xx и connect-level провалах; картинки/шрифты/медиа/CSS в браузере блокируются (нужен только текст).

Порты и деплой-матрица

Backend внутри контейнера всегда на `(8000)`, frontend на `(80)`. Наружу (на хост) пробрасываются разные порты для prod и dev:

	parser3 (prod)	parser3-dev (dev)
compose-файл	<code>(docker-compose.yml)</code>	<code>(docker-compose.dev.yml)</code>
<code>(ROOT_PATH)</code> / base	<code>(/parser3)</code>	<code>(/parser3-dev)</code>
контейнеры	<code>(parser3-backend)</code> , <code>(parser3-frontend)</code>	<code>(parser3-dev-backend)</code> , <code>(parser3-dev-frontend)</code>
хост-порт backend	8767 → 8000	8769 → 8000
хост-порт frontend	8768 → 80	8770 → 80
<code>(VITE_BASE_PATH)</code>	<code>(/parser3)</code> (дефолт)	<code>(/parser3-dev)</code>
сеть	дефолтная	именованная <code>(parser3-dev-net)</code> (alias <code>(parser3-backend)</code>)
каталог на сервере	в репе не зафиксирован	<code>(/opt/parser3-dev)</code>
код / образ	идентичны dev	идентичны prod

Порты `(3000)`, `(8765)`, `(8766)` к parser3 отношения не имеют. Vite dev-server по умолчанию слушает `(5173)`, а не `(3000)`. В системном nginx `(8765/8766)` — это `(parser2)`, `(8000/3000)` — иной сервис. Всё это вне данного репозитория.

Запуск

Прод и dev поднимаются **только через Docker Compose** (`(restart: unless-stopped)`). systemd-юнитов в репозитории нет; для прода голых процессов не предусмотрено.

Прод:

```
bash
```

```
docker compose build && docker compose up -d
```

Dev — через Makefile (обёртка над `docker-compose.dev.yml`):

```
bash

make up          # docker compose -f docker-compose.dev.yml up -d --build (8769/
make rebuild     # down + up --build
make down
make logs        # logs -f --tail=100
make check       # проверка конфига внутри контейнера, ждём: 8 180 True 8
```

Локально без Docker (для разработки):

```
bash

uvicorn backend.main:app --reload --port 8000 # backend
npm run dev                                # frontend (Vite, :5173)
```

Деплой на сервер (`213.171.17.139`), каталог `/opt/parser3-dev`) — порядок `git` важен:

```
bash

make fetch      # git fetch origin && git checkout <BRANCH> && git reset --hard
make rebuild
make check
```

Nginx: два уровня

nginx в проекте **два**, и это принципиально:

1. Контейнерный (`frontend/nginx.conf`)

Внутри frontend-образа. Раздаёт SPA и проксирует `/<base>/api/`, `/ws/`, `/health`, `/docs`, `/openapi.json` на backend-контейнер (`parser3-backend:8000`). Плейсхолдер `${VITE_BASE_PATH}` подставляется через `envsubst` в `entrypoint` при старте контейнера. Здесь же выставлен `client_max_body_size 50M` и таймауты (`proxy_read_timeout 300s` для API, `3600s` для WS).

2. Системный (host) nginx

Reverse-проxy на самом сервере: `/parser3/` → `127.0.0.1:8768`, `/parser3-dev/` → `127.0.0.1:8770` (frontend-контейнеры), а также прямые маршруты `/parser3-dev/api/`, `/ws/`, `/health`, `/docs` → backend-контейнер (`8769`). **Конфига системного**

nginx в репозитории нет — он живёт на сервере отдельно; в репе только сниппет в `README.md`.

⚠ **Открытый вопрос.** `NOTES/HANDOFF.md` §3 утверждает, что «контейнерный фронт не используется», хотя оба `compose` собирают `frontend`-контейнер, а `README` направляет на `:8768/parser3/`. Нужно свериться с реальным конфигом сервера: раздаётся ли статика фронт-контейнером (8768/8770) или напрямую системным `nginx`. Не додумано — помечено как противоречие.

HTTP Basic Auth (на системном nginx)

Пароль на приложение ставится **только на уровне системного nginx** — в самом приложении авторизации нет (`middleware` всего два: `CORS` и логирование запросов).

Файл с паролями:

```
bash

sudo apt install apache2-utils          # если нет htpasswd
sudo htpasswd -c /etc/nginx/.htpasswd <user> # -с ТОЛЬКО для первого юзера
sudo htpasswd      /etc/nginx/.htpasswd <user2> # добавить ещё — без -с
```

Директивы в защищаемые `location`:

```
nginx

auth_basic          "parser3-dev";
auth_basic_user_file /etc/nginx/.htpasswd;
```

⚠ **`auth_basic` не наследуется между `location`.** У субпути `parser3-dev` шесть блоков (`/health`, `/api/`, `/ws/`, `/docs`, `/openapi.json`, `/`). Пароль, повешенный только на `catch-all` `^~ /parser3-dev/`, оставит `/api/`, `/ws/`, `/docs`, `/openapi.json` **открытыми** — более специфичный `location` матчится первым и своего `auth_basic` не имеет. Чтобы не дублировать, удобно вынести пару директив в сниппет и подключать `include`:

```
nginx

# /etc/nginx/snippets/parser3-dev-auth.conf
auth_basic          "parser3-dev";
auth_basic_user_file /etc/nginx/.htpasswd;
```

```
nginx
```

```
location ^~ /parser3-dev/api/ {  
    include snippets/parser3-dev-auth.conf;  
    proxy_pass http://127.0.0.1:8769/api/;  
    # ...  
}  
# ...и так в каждый из шести блоков (либо оставить /health открытым для мониторинга)
```

После правок: `sudo nginx -t && sudo systemctl reload nginx`.

WebSocket отдельной настройки под auth не требует: если страница уже авторизована в браузере, handshake на `/ws/` уходит с теми же credentials.

Конфигурация и переменные окружения

Источник конфига — секция `environment:` в `compose`, а не `.env`. Файл `.env` приложением **не читается** (`pydantic-settings` с `extra="ignore"`, `.env` в `.gitignore`); `.env.example` в корне — исключительно справочный. Дефолты — в `backend/core/config.py` (класс `Settings`), перебиваются env из `compose`.

Секретов в конфиге нет — ни ключей, ни токенов, ни паролей.

Ключ	Назначение	Где задаётся
<code>ROOT_PATH</code>	префикс монтирования <code>((/parser3 /parser3-dev))</code>	compose
<code>TZ</code>	таймзона <code>((Europe/Moscow))</code>	compose
<code>CRAWLER_MAX_CONCURRENT</code>	число воркеров (8)	compose
<code>BROWSER_POOL_SIZE</code>	размер пула Chromium (8)	compose
<code>VITE_BASE_PATH</code>	base-путь фронта (build-arg + runtime <code>envsubst</code>)	frontend Dockerfile / dev compose
<code>DATA_DIR</code> , <code>RESULTS_DIR</code> , <code>LOG_DIR</code> , <code>SQLITE_DB_PATH</code>	пути внутри контейнера	дефолты / <code>.env.example</code>
<code>FETCH_USE_BROWSER</code>	вкл/выкл Playwright <code>((false))</code> для CI/ тестов)	config default
<code>CORS_ORIGINS</code>	CORS <code>((*)</code>)	config default
<code>CRAWLER_PAGE_TIMEOUT_SEC</code> , <code>CRAWLER_CONNECT_TIMEOUT_SEC</code> , <code>SITE_TOTAL_TIMEOUT_SEC</code> , <code>DNS_PRECHECK</code> , <code>DNS_TIMEOUT_SEC</code> , <code>CRAWLER_DELAY_MIN/MAX_SEC</code> , <code>BROWSER_BLOCK_RESOURCES</code> , <code>BROWSER_NETWORKIDLE_MS</code>	тюнинг краулера/ браузера	дефолты config, env-override

Прод-значения таймаутов (из config, подтверждены `make check` → `(8 180 True 8)`): пер-сайт жёсткий `(site_total_timeout_sec=180)`, read `(crawler_page_timeout_sec=25)`, connect `(crawler_connect_timeout_sec=8)`, DNS `(dns_timeout_sec=5)`; воркеров 8, пул браузера 8, задержка между запросами `(0.3-1.0 c)`.

Тома и персистентность

- `./data:/app/data` — SQLite `(parser3.db)` (WAL), каталог `(uploads/)`, `(logs/)`.
- `./results:/app/results` — сгенерированные XLSX.
- `data/` и `results/` — в `(.gitignore)`.

- Логи фактически идут в **stdout** (`structlog` → `docker logs`); `LOG_DIR` в коде не используется (артефакт).
-

Справочник API

FastAPI, `root_path` из `ROOT_PATH`, REST под `/api/v1`. Swagger доступен на `/<root>/docs`, спека — `/<root>/openapi.json` (генерируется на лету, статического файла в репе нет).

Метод	Путь (после (/api/v1))	Назначение	Тело / параметры
POST	/tasks	Создать задачу из списка URL	{urls: string[], mode="all_contacts", target_positions?: string[]}
POST	/tasks/upload	Создать задачу из файла	multipart: {file} (.csv/.txt), {mode}, {target_positions}
GET	/tasks	Список задач (лимит 100)	—
GET	/tasks/{id}	Задача + её сайты	—
GET	/tasks/{id}/contacts	Контакты задачи	—
GET	/tasks/{id}/input	Скачать исходный загруженный файл	—
POST	/tasks/{id}/pause	Пауза	—
POST	/tasks/{id}/resume	Продолжить	—
POST	/tasks/{id}/cancel	Отмена	—
DELETE	/tasks/{id}	Удалить задачу (+contacts+sites)	—
GET	/tasks/{id}/download	Скачать XLSX	—
GET	/tasks/{id}/download/csv	Скачать CSV (лист «Все контакты»)	—
GET	/catalog/positions	Каталог должностей	—
GET	/blacklist	Список ЧС	—
POST	/blacklist	Добавить в ЧС	{entry_type: position fio email domain, value, source="user"}
DELETE	/blacklist/{id}	Удалить из ЧС	—

Метод	Путь (после <code>/api/v1</code>)	Назначение	Тело / параметры
GET	<code>/health</code> (вне <code>/api/v1</code>)	Healthcheck	—
WS	<code>/ws/{id}</code> (вне <code>/api/v1</code>)	Прогресс real-time	—

WebSocket `/ws/{id}`

При коннекте сразу приходит `snapshot`, далее сообщения по мере обработки, `keepalive` `ping` каждые 30 с. Типы (`msg.type`):

Тип	Полезная нагрузка
<code>snapshot</code>	<code>{task:{...}}</code> — при подключении
<code>ping</code>	keepalive (если 30 с нет событий)
<code>progress</code>	<code>{task_id, status:"running", stage:"fetching" "extracted" "exporting", current_url, site_current, processed, done, total, eta_seconds, found_contacts, sites_ok, sites_error}</code> — дважды на сайт
<code>completed</code>	<code>{task_id, output_file, found_contacts}</code>
<code>cancelled</code>	<code>{task_id}</code>
<code>failed</code>	<code>{task_id, error}</code>

Статусы задачи: `pending, running, paused, cancelled, completed, failed`. Статусы сайта: `pending, processing, ok, partial, error`.

Сборка

Backend (`Dockerfile`, `python:3.11-slim`): системные libs для Chromium → `pip install -r backend/requirements.txt` → `playwright install chromium` → копирование `backend/`. Запуск:

```
uvicorn backend.main:app --host 0.0.0.0 --port 8000 --proxy-headers --forwarded-allow-ips=*
```

Ключевые зависимости: `fastapi 0.115.4`, `uvicorn[standard] 0.32.0`, `pydantic 2.9.2`, `httpx 0.27.2`, `aiosqlite 0.20.0`, `beautifulsoup4`, `lxml`, `pymorphy3` (+dicts-

ru), (petrovich), (rapidfuzz), (phonenumbers), (openpyxl), (structlog), (playwright 1.47.0), (websockets).

Frontend ((frontend/Dockerfile), multi-stage): (node:20-alpine) → (npm install) → (npm run build) ((tsc -b && vite build)) → статика в (nginx:alpine) под (/usr/share/nginx/html\${VITE_BASE_PATH}); entrypoint через (envsubst) подставляет (VITE_BASE_PATH) в nginx-конфиг.

(package.json) scripts:

```
json

"scripts": {
  "dev": "vite",
  "build": "tsc -b && vite build",
  "preview": "vite preview"
}
```

Инициализация при старте backend ((lifespan)): (Database.connect()) → (migrate()) (идемпотентный SQL из (backend/storage/migrations/*.sql)) → (TaskManager.start()) (поднимает (BrowserPool) + (Fetcher), сбрасывает зависшие (running) → (paused) и (processing) → (pending)).

Чего в репозитории нет (сводка «не додумывать»)

- **parser2** и любые ИИ/LLM-компоненты — нет (только нереализованная бэклог-идея (T2) «LLM-fallback» в (NOTES/HANDOFF.md)).
- **Сущность/режим «РОССИИ»** — нет; в коде клиент зовётся «РФОП».
- **Конфиг системного (host) nginx** — нет (только сниппет в README).
- **systemd-юниты** — нет; запуск только через Docker Compose.
- **Порты 3000 / 8765 / 8766** — в этом репозитории не используются.
- **Путь прод-каталога на сервере** — в репе не зафиксирован (есть только dev = (/opt/parser3-dev)).
- **Статический OpenAPI-файл** — нет; спека генерируется FastAPI на лету.
- **Противоречие для проверки:** «контейнерный фронт не используется» (HANDOFF) против наличия frontend-контейнера в обоих compose.